

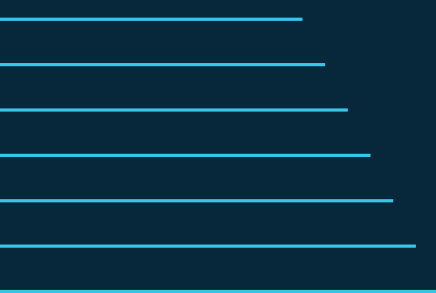
COMP3020

TMDB API Setup and Statistical Analysis in R

A visual guide to creating developer access, installing tmdbR, collecting a bounded movie dataset, and describing it responsibly in RStudio.

REGISTER	CONNECT	DESCRIBE	INTERPRET
TMDB account + API	tmdbR 0.2.0	plots + summaries	limits + bias

Never place a real API token in code, screenshots, reports, or Git.



START HERE

Your 45-minute setup path

Complete the first seven steps once. The statistics workflow can then be reused for other questions and assignments.

1 Create account Join TMDB on a desktop browser and verify your email.	2 Request API Settings > API > Request an API Key > Developer.	3 Describe use Enter honest educational, non-commercial application details.
4 Copy token Use the long API Read Access Token, not a session token.	5 Install R tools Install R, RStudio, and the supplied tmdbR archive.	6 Cache token Run <code>tmdb_auth()</code> once; the credential stays outside scripts.
7 Test access Request one known movie and check the returned title.	8 Ask a question Define a population, variables, and a bounded dataset.	9 Describe carefully Summarise, visualise, and state important limitations.

Before you begin

- Use a desktop browser; TMDB's API registration flow is not designed for mobile.
- Install [R 4.1.0 or later](#) and the current stable [RStudio Desktop](#).
- Keep the supplied `tmdbR_0.2.0.tar.gz` file zipped.
- Use truthful account and application information; you will accept TMDB's terms.

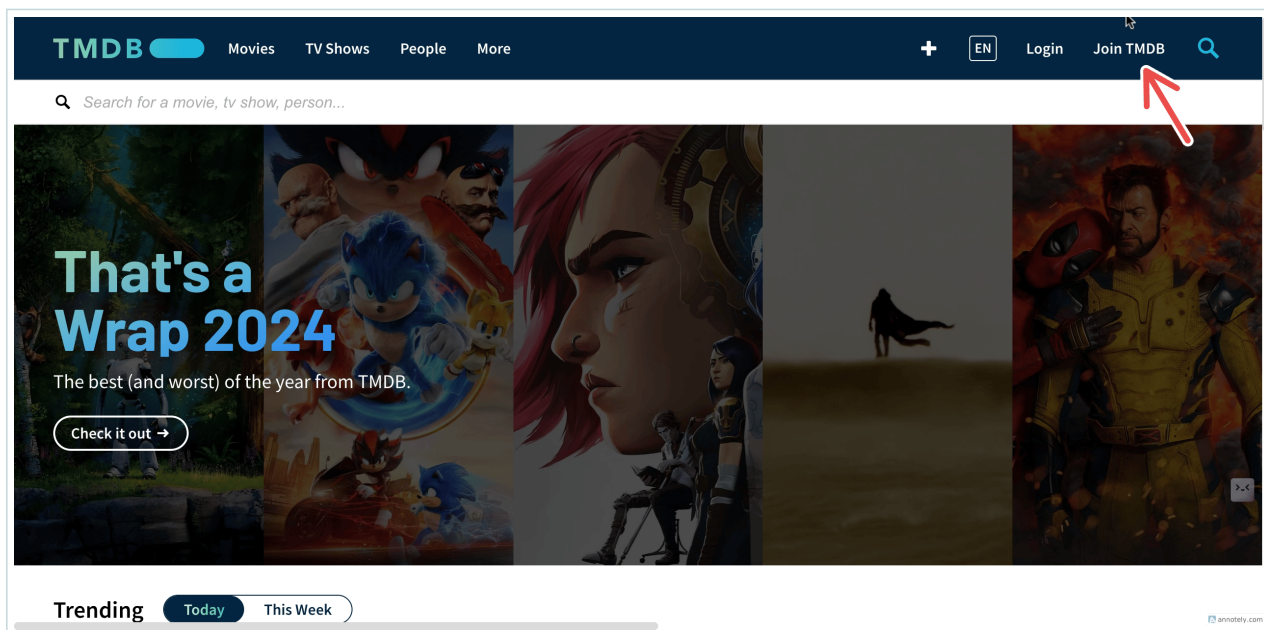
Possible extension

If an assignment permits an open-ended approach, explore TMDB fields, define a question, and choose methods that match the variables, assumptions, and research aim. The chi-square example is one possibility.

01 / TMDB ACCOUNT

Join and verify your account

Account creation is free. Use your own email address and keep the password private.



Real walkthrough screenshot: select Join TMDB in the top-right corner. Source: TMDB Quickstart mirror (accessed 8 August 2026).

1

Open sign-up

Go to themoviedb.org/signup or select Join TMDB.

2

Enter details

Create a username and strong, unique password.

3

Verify email

Open TMDB's message and activate your account.

Then sign in

- If the verification message is missing, check junk mail and confirm that your email was typed correctly.
- Return to TMDB and sign in. Your profile icon will appear in the top-right corner.
- Do not use disposable contact details: the API application asks for genuine information.

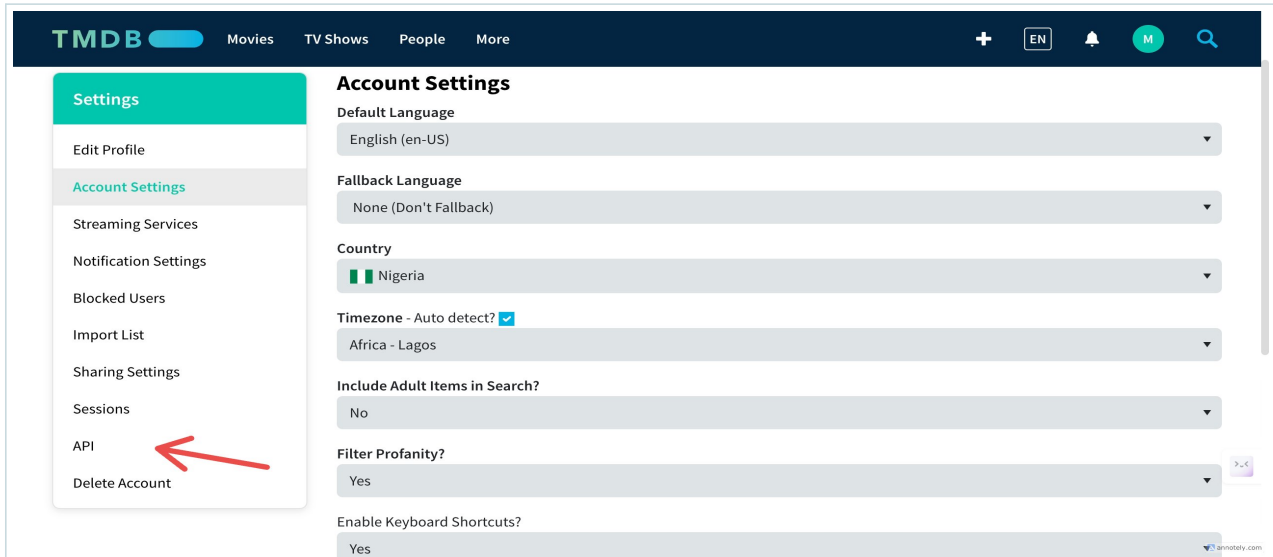
Direct link

<https://www.themoviedb.org/signup>

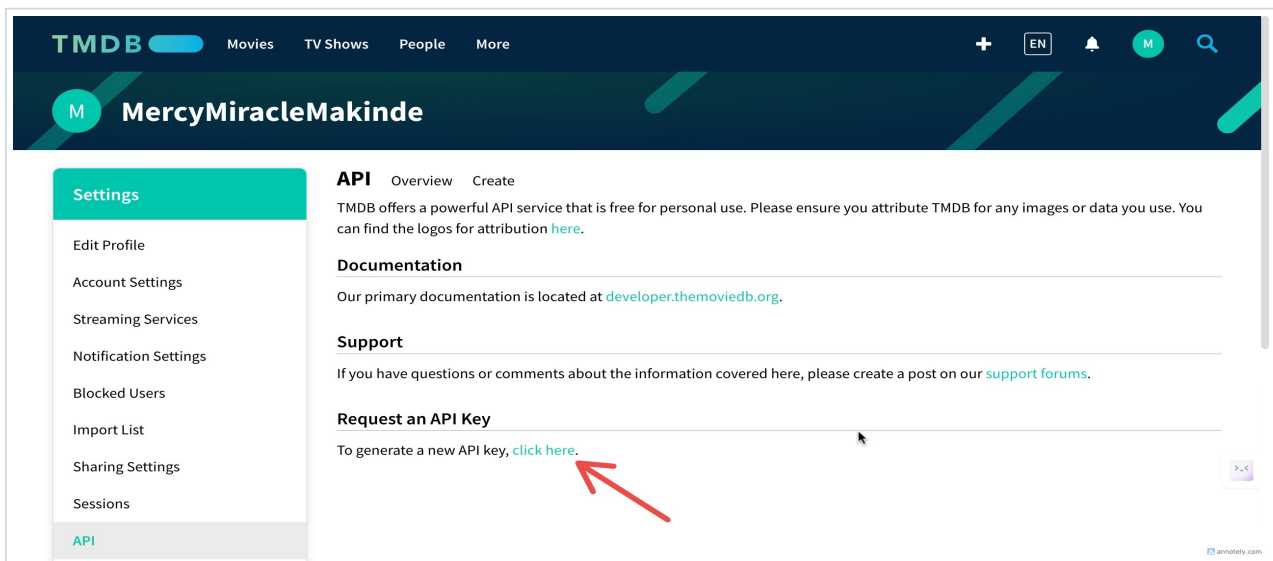
02 / API ACCESS

Open API settings and request a key

The API menu is inside Account Settings. If you have never applied before, the page shows a Request an API Key section.



Select API in the Settings sidebar. Screenshot source: TMDB Quickstart mirror; example account details belong to the source author.



Under Request an API Key, select the 'click here' link. The interface may move, but the wording remains the key landmark.

Choose Developer

Select the Developer application type for coursework and personal experiments. Read and accept TMDB's terms yourself; do not select Professional unless the intended use is genuinely commercial.

Signed-in shortcut: themoviedb.org/settings/api

03 / APPLICATION DETAILS

Describe the course project honestly

The current Details page uses three application fields. Contact fields may also appear during first registration.

Working example

These values are copied from the instructor's accepted TMDB application Details page. Enter them exactly as shown for the COMP3020 course setup.

Field	Working COMP3020 value	Purpose
Application Name	COMP3020	Identifies the course application.
Application URL	http://127.0.0.1	Local course application address accepted in the working setup.
Application Summary	Test app for Western Sydney University COMP3020 Unit.	States the educational purpose concisely.

After approval, return to Settings > API. Copy the long API Read Access Token. The source screenshot intentionally blurs both credentials.

Use this	Not this
API Read Access Token Long bearer token used by <code>tmdb_auth()</code> .	Request token or session ID Used for user-approved write operations; tmdbR is read-only.
API Key (v3 auth) Supported alternative through <code>TMDB_API_KEY</code> .	A token copied into an R script, report, screenshot, or Git repository.

Credential rule

Never show the token to another person. If it is exposed, regenerate it in TMDB settings and replace the cached value.

04 / RSTUDIO

Install tmdbR in three clear steps

Use the supplied release archive exactly as downloaded. RStudio will install it into your personal R package library.

File you need

tmdbR_0.2.0.tar.gz - keep it zipped and save it somewhere easy to find, such as Downloads or the course folder.

Step 1 - Open the RStudio Console

- Start RStudio and wait until the blue > prompt appears in the Console pane.
- Do not open, extract, or double-click the archive. The installation command reads it directly.

Step 2 - Choose and install the archive

```
package_file <- file.choose()
install.packages(package_file, repos = NULL, type = "source")
```

What happens

A file chooser opens after the first line. Select tmdbR_0.2.0.tar.gz and choose Open. Then run the second line. Wait until the Console returns to the > prompt.

Step 3 - Load the package and verify the version

```
library(tmdbR)
packageVersion("tmdbR")
# Expected output: [1] '0.2.0'
```

If you see...	Meaning / action
[1] '0.2.0'	Installation succeeded. Continue to authentication on the next page.
Package is not available	Check that you selected the supplied archive, not a folder or an unzipped file.
Compilation tools are missing	Windows: install matching Rtools. macOS: install Apple's Command Line Tools. Restart RStudio and retry.

Remember

Install once per R library. In every new R session, load the installed package with `library(tmdbR)`.

05 / AUTHENTICATION

Cache the bearer token once

tmdb_auth() stores an encrypted token file in your user cache, outside the project and assignment scripts.

```
library(tmdbR)
tmdb_auth()
# Paste the long API Read Access Token only at the prompt
```

1

Prompt appears

Paste the long token, then press Enter.

2

Cache is written

The file is outside the course project.

3

Future requests

tmdbR finds the cached token automatically.

Test with one known movie

```
test <- movie(id = 550, language = "en-AU")
test$title
# Expected: "Fight Club"
```

Task	Command	Safe result
Locate cache	tmdb_token_path()	Shows only the file path.
Replace token	tmdb_auth(overwrite = TRUE)	Prompts for a replacement.
Remove token	tmdb_forget_token()	Deletes the cached credential.

401 error?

Copy the API Read Access Token again, check for leading/trailing spaces, and run tmdb_auth(overwrite = TRUE). Never paste the token into a screenshot sent for help.

06 / FINDING RECORDS

Search first, then request details by ID

Names and titles are not unique. TMDb IDs are the reliable link between searches, movie details, people, and credits.

1. Search and inspect candidate rows

```
# Search by title, then display three rows as a checkpoint
hits <- search_movie(query = "Spirited Away", language = "en-AU")
head(hits$results[c("id", "title", "release_date",
                    "vote_average", "vote_count")], 3)
```

ID	Title	Release date	Rating	Votes
129	Spirited Away	2001-07-20	8.534	18,651
1450777	Uncovering Spirited Away	2024-07-07	7.000	1
698296	The Art of 'Spirited Away'	2003-04-15	8.167	6

2. Select the correct ID and request related records

```
# Select the exact title and reuse its stable TMDb ID
chosen <- subset(hits$results, title == "Spirited Away")
movie_id <- chosen$id[[1]]
details <- movie(id = movie_id, language = "en-AU")
credits <- movie_credits(id = movie_id)
# Display the fields used as output checks
details[c("title", "runtime", "budget", "revenue")]
head(credits$cast[c("name", "character")], 3)
```

Details field	Sample value	Cast name	Character
title	Spirited Away	Rumi Hiiragi	Chihiro (voice)
runtime	125 minutes	Miyu Irino	Haku (voice)
budget	19,000,000	Mari Natsuki	Yubaba / Zeniba (voice)
revenue	274,925,095		

Expected variation: the movie ID and title should match; ratings, vote counts, revenue, and result ordering can change as TMDb updates its data.

07 / WORKED STATISTICS EXAMPLE

A six-step chi-square workflow

Use one question, one contingency table, and one test. The same six-step structure can be reused for other pairs of categorical variables.

1

Ask

Write the research question.

2

Hypothesise

State H_0 and H_A before testing.

3

Count

Build the 3 x 2 table.

4

Check

Inspect independence and expected counts.

5

Test

Run `chisq.test()`.

6

Conclude

Use p and context to answer.

1. Research question

Among currently popular Australian-market movies with at least 50 votes, is the proportion rated 7.5 or higher associated with release era?

2. State the hypotheses

Hypothesis	Statement
H_0	The distribution of rating category is independent of release era in the population represented by this sampling process.
H_A	The distribution of rating category differs across at least one release era.

Prepare the data with the R 4.1 pipe

```
# Download a bounded set of popular Australian-market movies
response <- movie_popular(
  region = "AU", language = "en-AU", paginate = TRUE,
  max_pages = 5, max_results = 100, deduplicate_by = "id"
)

# Begin with the movie rows stored in the API response
movies <- response$results |>
# Add a new variable to the data frame
transform(
  # Extract the four-digit year from the release-date text
  release_year = as.integer(substr(release_date, 1, 4))
) |>
# Retain movies with 50+ votes and a known release year
subset(vote_count >= 50 & !is.na(release_year)) |>
# Add the two categorical variables required by the test
transform(
  # Divide release year into three mutually exclusive eras
  release_period = cut(
    # -Inf and Inf include every possible year at the ends
    release_year, c(-Inf, 2009, 2019, Inf),
    # Apply readable labels in the same order as the intervals
    labels = c("Before 2010", "2010-2019", "2020 or later")
  ),
  # Convert the numerical rating into two categories
  rating_category = ifelse(
    # TRUE receives the first label; FALSE receives the second
    vote_average >= 7.5, "7.5 or higher", "Below 7.5"
  )
)

# Checkpoint: sort high to low, select columns, inspect three rows
preview <- movies[order(movies$vote_average, decreasing = TRUE),
  c("title", "release_year", "vote_average",
    "release_period", "rating_category")]
head(preview, 3)
```

08 / STEPS 3 AND 4

Count first, then check conditions

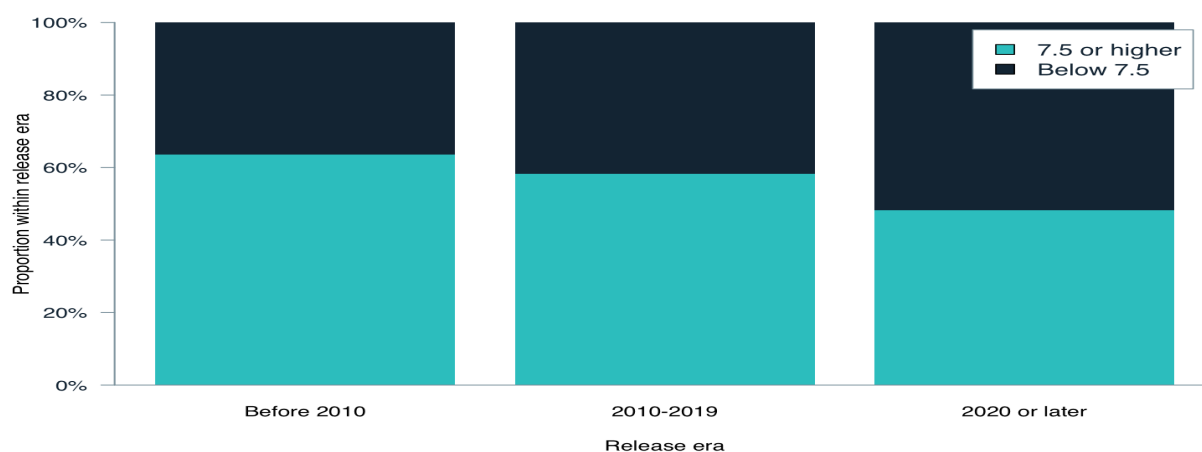
After applying the 50-vote rule, 79 movies remained in the worked example.

Checkpoint: inspect the two analysis variables

Movie	Year	Rating	Release period	Rating category
Avatar Aang: The Last Airbender	2026	9.300	2020 or later	7.5 or higher
Swapped	2026	8.888	2020 or later	7.5 or higher
The Shawshank Redemption	1995	8.727	Before 2010	7.5 or higher

3. Build the observed-count table

Release era	Rated 7.5 or higher	Rated below 7.5	Row total
Before 2010	7	4	11
2010-2019	7	5	12
2020 or later	27	29	56
Column total	41	38	79



Compare the proportions within each release-era group before reading the test result.

4. Check expected counts if H0 is true

Release era	Rated 7.5 or higher	Rated below 7.5
Before 2010	5.71	5.29
2010-2019	6.23	5.77
2020 or later	29.06	26.94

Condition	Check
Count data	Each movie contributes to exactly one cell.
Independent observations	No duplicate movie IDs were retained.
Expected counts	Smallest expected count = 5.29; all six are at least 5.

09 / STEPS 5 AND 6

Test, decide, and answer the question

chisq.test() performs Pearson's chi-square test on this 3 x 2 contingency table.

5. Run the test and check its output

```
# Step 3: create and display the observed-count table
counts <- with(movies, table(release_period, rating_category))
counts
# Step 4: test, then check that expected counts are at least 5
fit <- chisq.test(counts)
round(fit$expected, 2)
# Step 5: keep the three statistics needed for reporting
fit[c("statistic", "parameter", "p.value")]
# Step 6: compare p with the chosen significance level
alpha <- 0.05
if (fit$p.value < alpha) "Reject H0" else "Fail to reject H0"
```

79

MOVIES

1.111

CHI-SQUARE

2

DF

0.5739

P-VALUE

6. Conclude	Interpretation
Compare	$p = 0.5739$ is greater than $\alpha = 0.05$.
Decision	Fail to reject H_0 ; the result is not statistically significant at the 5% level.
Answer	This sample does not provide sufficient evidence that rating category differs across the three release eras.

Model report

A chi-square test of independence found insufficient evidence of an association between release era and rating category, $X^2(2, N = 79) = 1.111, p = 0.5739$.

What this result does not mean

- It does not prove that the variables are independent; it only says this sample did not provide sufficient evidence at $\alpha = 0.05$.
- It does not show causation. Release era cannot be said to cause a movie to cross the rating threshold.
- Changing thresholds after seeing the data would change the question and may inflate false-positive risk.
- TMDB's popular list changes over time, so record the retrieval date and expect counts to differ on another day.

Extension

For a formal assignment, justify the categories before analysis and discuss whether the popularity-ranked sample represents the population named in your research question.

KEEP NEARBY

Package help and function categories

Use R's help system to inspect arguments and examples. The directory below highlights the functions students are most likely to need.

Open help from the RStudio Console

```
# Open the package index and browse all documented topics
help(package = "tmdbR")
# Open one function help page
?search_movie
# Display its formal arguments
args(search_movie)
# Search package help for a word or phrase
help.search("credits", package = "tmdbR")
# List functions after library(tmdbR) has been run
ls("package:tmdbR")
```

Common functions by category

Category	Useful functions
Authentication and settings	tmdb_config(), tmdb_credentials(), tmdb_auth(), tmdb_token_path(), tmdb_forget_token()
Requests and pagination	tmdb_request(), tmdb_request_all(), tmdb_paginate()
Search	search_movie(), search_person(), search_multi(), search_keyword(), search_tv()
Movies	movie(), movie_credits(), movie_keywords(), movie_reviews(), movie_similar(), movie_popular(), movie_topRated(), movie_now_playing()
Television	tv(), tv_credits(), tv_episode(), tv_season(), tv_popular(), tv_topRated()
People	person_tmdb(), person_movie_credits(), person_combined_credits(), person_popular()
Discovery and reference	discover_movie(), discover_tv(), genres_movie_list(), genres_tv_list(), configuration(), find_tmdb()

KEEP NEARBY

Troubleshooting and sources

Check installation, authentication, request limits, and response structure before changing an analysis.

Fast troubleshooting

Symptom	Action
Package will not install	Check archive path and platform build tools; restart RStudio.
Missing credentials / 401	Run <code>tmdb_auth()</code> ; replace the cache if the token was copied incorrectly.
HTTP 429	Wait and reduce pages. Keep sequential requests and bounded limits.
Empty results	Relax filters, verify IDs, and inspect the first response page.
Nested or irregular fields	Use <code>names()</code> , <code>str()</code> , or <code>simplify = FALSE</code> before analysis.

Before submission Remove credentials; record retrieval date, endpoint, filters, rows, and truncation; label plots; distinguish description from causal explanation; attribute TMDB.

Sources and further help

Official: [TMDB application authentication](#); [TMDB FAQ and attribution](#); [signed-in API settings](#). Visual walkthrough screenshots: [TMDB Quickstart mirror](#), accessed 8 August 2026. Package help: `help(package = "tmdbR")`. Advanced exercise: `scripts/mcu_actor_network.R`.